



SYSTEM DEVELOPMENT SPECIFICATION

MR. MUTIA MUTINDA
SUPERVISOR

philip lee
22/07341

1. INTRODUCTION

1.1 Purpose of the SDS

This Software Design Specification (SDS) describes the detailed design of the FRACA SERVCOM Inventory Management System. It outlines the system architecture, module designs, data structures, interfaces, and constraints that guide developers during implementation.

1.2 Scope

The SDS covers the system components needed to support product management, stock tracking, purchase and sales records, low-stock alerts, reporting, and user role management.

1.3 Definitions and Acronyms

- **IMS** – Inventory Management System
- **DBMS** – Database Management System
- **UI** – User Interface
- **CRUD** – Create, Read, Update, Delete

2. System Overview

The FRACA SERVCOM IMS is a web-based system built using **PHP, MySQL, HTML, CSS, JavaScript**, with possible use of **Laravel** as the backend framework. The system automates stock management and reporting for a medium-sized hardware and furniture business.

3. System Architecture

3.1 Architectural Style

- **Three-tier architecture:**
 - *Presentation Layer:* HTML, CSS, JavaScript
 - *Application Layer:* PHP or Laravel
 - *Data Layer:* MySQL database

3.2 System Architecture Diagram

(Insert diagram here later)

4. Detailed Design

4.1 Modules Overview

The system consists of the following modules:

1. Product Management
2. Stock Control
3. Sales & Purchase Management
4. Low-Stock Alerts
5. Reporting
6. User Management
7. Authentication Module

4.2 Module 1: Product Management

Functions:

- Add new products
- Update product details
- Delete or deactivate products
- View product list

Inputs: Product name, category, barcode, quantities, threshold, price.

Outputs: Product registration confirmation, error messages, product list.

Database Tables Used: products

Major Processes: Data validation → Insert/Update → Confirmation.

4.3 Module 2: Stock Control

Functions:

- Update stock (increase/decrease)
- Track stock levels
- Generate stock status

Database Tables Used: products, stock_history

Processes: CRUD on stock entries, automatic updates to stock table.

4.4 Module 3: Sales & Purchase Management

Functions:

- Record sales
- Record purchases
- Auto-update stock on each transaction

Database Tables Used: sales, purchases, products

4.5 Module 4: Low-Stock Alerts

Functions:

- Detect products with stock lower than threshold
- Send visual alerts (dashboard, email optional)

Database Tables Used: products

4.6 Module 5: Reporting

Functions:

- Stock report
- Sales report
- Purchase report
- Daily statistics

Outputs: Exportable/report views.

4.7 Module 6: User Management

Roles: Admin, Staff.

Functions: Create users, assign roles, deactivate users.

Database Table: users

4.8 Module 7: Authentication

Functions: Login, logout, password hashing, session control.

Database Table: users

5. Database Design

5.1 ER Diagram

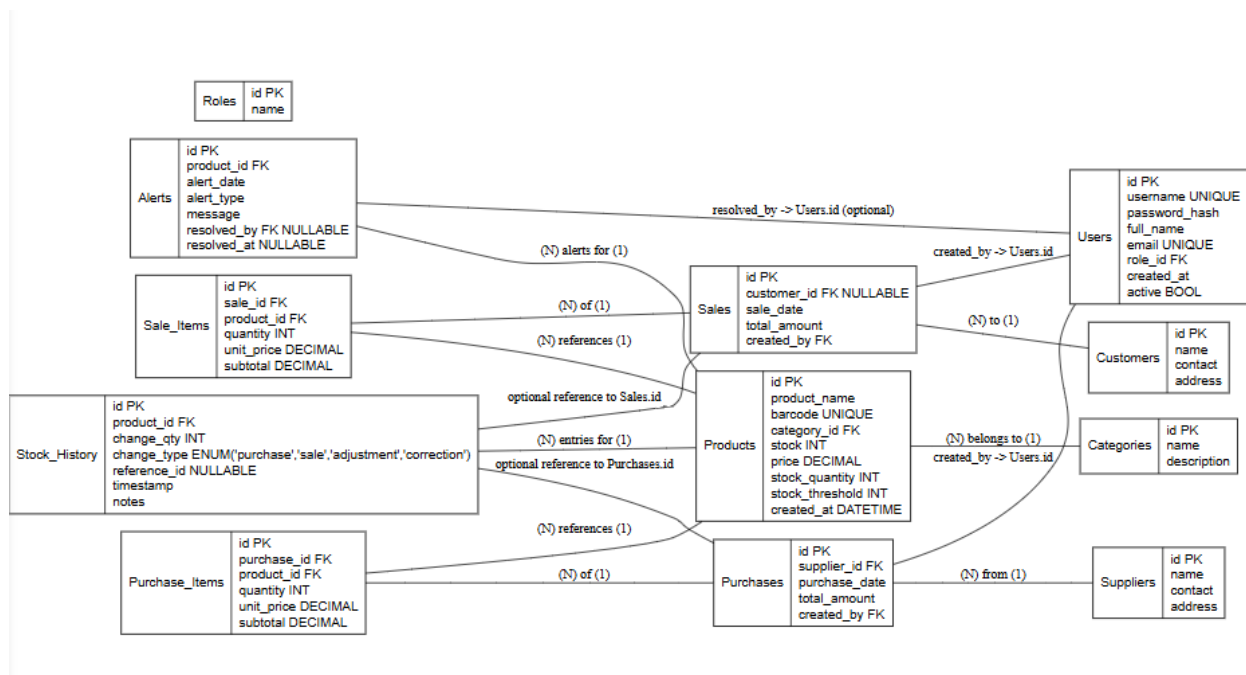
A complete, implementation-ready ERD has been added here. The ERD includes:

- Detailed entity definitions (attributes, primary keys, unique constraints, nullable fields).
- Foreign keys and relationship cardinalities.
- SQL DDL (CREATE TABLE) statements for all main tables so developers can create the schema immediately.

Entities included: products, categories, users, roles, suppliers, customers, purchases, purchase_items, sales, sale_items, stock_history, and alerts.

Notes on design choices:

- sales and purchases are master tables with corresponding _items child tables to represent line items and support many-to-many relationships between transactions and products.
- categories is normalized to avoid string duplication in products and to enable category-level reporting.
- stock_history records all stock movements and stores a reference to the source transaction (sale/purchase/adjustment) for traceability.
- users and roles implement role-based access control; created_by fields in transactions track who performed the action.



SQL DDL (sample, ready to run with minor adjustments):

```
CREATE TABLE categories (  
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id),  
    FOREIGN KEY (created_by) REFERENCES users(id)  
);
```

```
CREATE TABLE purchase_items (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    purchase_id INT,  
    product_id INT,  
    quantity INT DEFAULT 0,  
    unit_price DECIMAL(12,2) DEFAULT 0.00,  
    subtotal DECIMAL(12,2) AS (quantity * unit_price) STORED,  
    FOREIGN KEY (purchase_id) REFERENCES purchases(id),  
    FOREIGN KEY (product_id) REFERENCES products(id)  
);
```



```
CREATE TABLE sales (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NULL,  
  sale_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  total_amount DECIMAL(12,2) DEFAULT 0.00,  
  created_by INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(id),  
  FOREIGN KEY (created_by) REFERENCES users(id)  
);
```

```
CREATE TABLE sale_items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  sale_id INT,  
  product_id INT,  
  quantity INT DEFAULT 0,  
  unit_price DECIMAL(12,2) DEFAULT 0.00,  
  subtotal DECIMAL(12,2) AS (quantity * unit_price) STORED,  
  FOREIGN KEY (sale_id) REFERENCES sales(id),  
  FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

```
CREATE TABLE stock_history (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  product_id INT,  
  change_qty INT NOT NULL,  
  change_type VARCHAR(32) NOT NULL,  
  reference_id INT NULL,  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
  notes TEXT,  
  FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

```
CREATE TABLE alerts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  product_id INT,  
  alert_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  alert_type VARCHAR(50),  
  message TEXT,  
  resolved_by INT NULL,  
  resolved_at DATETIME NULL,  
  FOREIGN KEY (product_id) REFERENCES products(id),  
  FOREIGN KEY (resolved_by) REFERENCES users(id)  
);
```

5.2 Main Tables

- products
- users
- sales
- purchases
- stock_history

6. Interface Design

6.1 User Interface Description

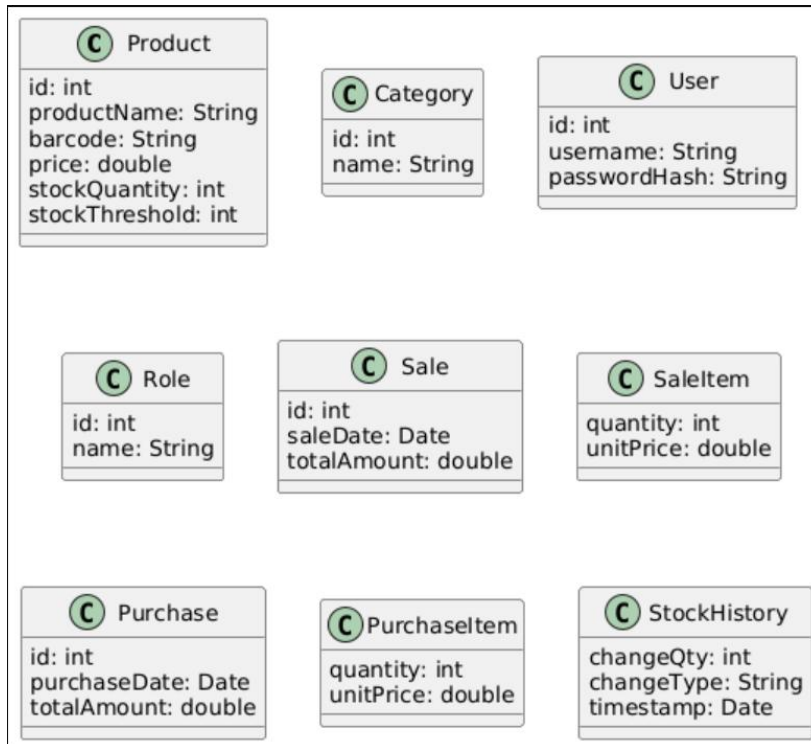
- Dashboard
- Product form pages
- Stock management pages
- Sales and purchase forms
- Reports page
- User admin pages

7. Security Requirements

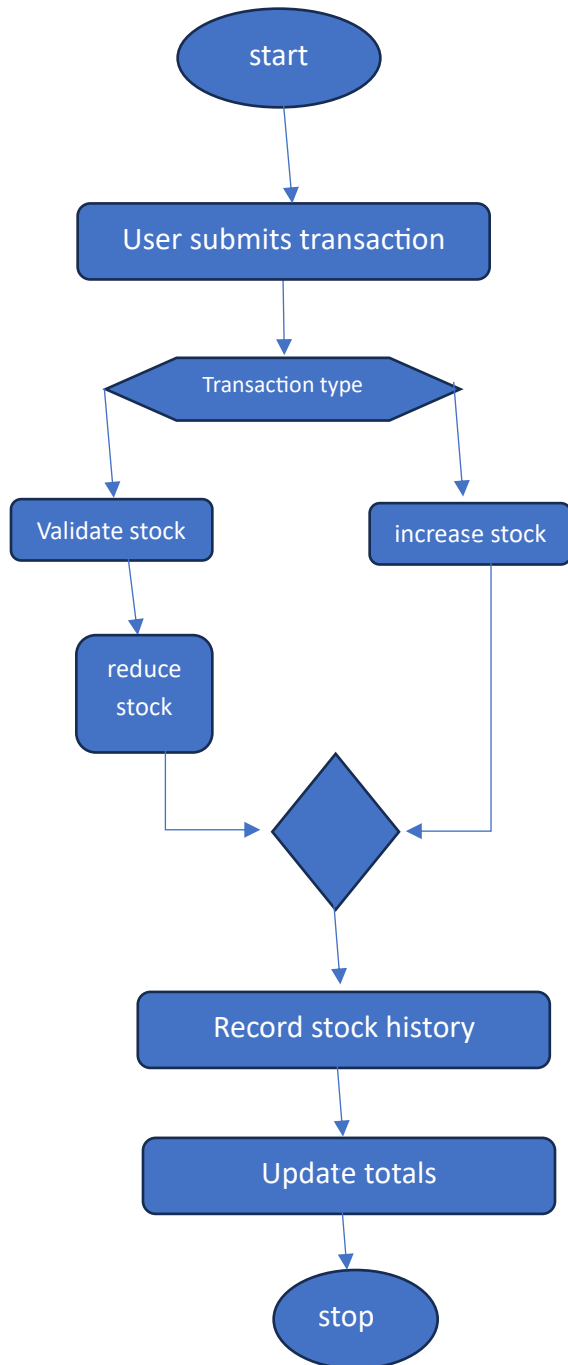
- Password hashing
- Role-based access
- Input sanitization
- CSRF protection (if using Laravel)

8. UML Design Models and Other diagrams

8.1 UML Class Diagram



8.2 Activity diagram/Flowchart



9. **Non-Functional Design**

9.1 **Performance**

Should handle 100+ products with minimal delay.

8.2 **Usability**

Clear forms

Mobile-responsive layout

8.3 **Reliability**

Daily backup

8.4 **Scalability**

Easy to add modules in the future.

9. **Constraints**

- Must run on XAMPP (PHP + MySQL)
- Browser-based
- Budget constraints favor lightweight solutions

10. Appendices

Sample SQL table definitions

UML diagrams

Flowchart